

1) Perform Data Cleansing:

Cleansing: removes duplicates, handles missing values (median for numeric, mode for categorical), caps outliers with IQR.

✓ Explanation of each step and its output

➤ Duplicate Removal

Duplicates removed. Dataset shape: (255347, 18)

- It found and removed duplicate rows in the dataset.
- Now, the dataset has **255,347 rows and 18 columns** (features + target).
- Removing duplicates ensures that repeated data doesn't skew your analysis.

➤ Handling Missing Values

For each column, it filled missing values as follows:

➤ Numeric columns:

- **Median** is used because it's less affected by extreme values (outliers).

Examples:

- Age: Missing values filled with **43.0**
- Income: Missing values filled with **82466.0**
- LoanAmount: Filled with **127556.0**

➤ Categorical columns:

- **Mode** (most frequent value) is used because it's a representative choice for missing categories.

Examples:

- Education: Filled with **Bachelor's**
- EmploymentType: Filled with **Part-time**
- HasMortgage: Filled with **Yes**

This ensures there are no blank or null entries, making the data ready for further analysis.

➤ Handling Outliers

Outliers are values that are unusually high or low and can affect model performance.

The **IQR (Interquartile Range)** method was used:

- Values below $Q1 - 1.5 \times IQR$ are capped to the lower bound.
- Values above $Q3 + 1.5 \times IQR$ are capped to the upper bound.

Output example:

- Outliers capped in Default: lower=0, upper=29653
→ 29,653 outliers above the upper bound were capped.

For other columns like Age, Income, etc., it found **no outliers** to cap, so lower=0 and upper=0 means no changes were necessary.

✅ Final Outcome:

1. **Duplicates removed** → cleaned dataset.
2. **Missing values filled** → no blanks or NaNs remain.
3. **Outliers handled** → extreme values are managed to prevent bias.

2) Imputation:

➤ Missing values BEFORE imputation

Missing values BEFORE imputation:

Series([], dtype: int64)

- This means that at this point, **there are no missing values** in any column.
- Why? Because in your earlier **Data Cleansing** step, you already handled all missing data — by filling missing numeric values with the median and categorical ones with the mode.
- So when you check before imputation, nothing is left to fill!

➤ Imputation steps executed

Missing numeric values imputed with median.

Missing categorical values imputed with mode.

- The code ran the imputation anyway, even though it wasn't necessary this time.
- It applied the median and mode strategies to all columns but since there were no missing values, nothing actually changed.

➤ Missing values AFTER imputation

Missing values AFTER imputation:

✅ No missing values remain!

- After attempting imputation, the dataset still has no missing values.
- This confirms that all missing entries (if any) were already filled earlier.

➤ Example rows where missing numeric values were present

Example rows where missing numeric values were present:

Empty DataFrame

Columns: [...]

Index: []

- This confirms that there are **no rows with missing numeric values** left.
- Since cleansing already handled them, the DataFrame is complete.

✅ **Final takeaway:**

- **The output is correct** because your earlier cleansing step already removed or filled all missing values.
 - Running imputation again didn't change anything but it confirmed that no missing data exists now.
 - It's like checking a cleaned room — everything is already in place!
-

3) Encoding:

Encoding is the process of converting categorical (non-numeric) data into numeric format so that machine learning algorithms can process it.

✅ **Two common encoding methods:**

1. **One-Hot Encoding** – Creates binary columns for each category.
2. **Label Encoding** – Assigns an integer to each category.

For this dataset, **One-Hot Encoding** is generally preferred because it avoids implying any order between categories.

📊 **Why so many columns?**

- Some categorical columns have many unique values (like LoanPurpose, Education, etc.).
- One-hot encoding creates a separate column for each unique category.
- Example: If LoanPurpose has 100 unique values, it creates 100 columns!
- This is why your dataset ballooned from **18 columns → 10,024 columns**.

Step 1. Dataset

- You had **10,000 rows** and **18 columns**.
- Some columns were **numbers** (like Income, Age, LoanAmount).
- Some columns were **categories** (like Education, EmploymentType, MaritalStatus, etc.).

Step 2. Encoding rules

- If a column had **too many unique values** (>50), we used **Label Encoding**.
- If a column had **small unique values** (≤50), we used **One-Hot Encoding**.

Step 3. Encoding results

- **LoanID** → had **10,000 unique values** (every loan is different).
→ Converted into integers (Label Encoded).

(This column is usually just an ID and not useful for prediction, but encoding handled it anyway.)

- **Education, EmploymentType, MaritalStatus, HasMortgage, HasDependents, LoanPurpose, HasCoSigner** →

These had only 2–5 unique categories.

→ Converted into extra columns (One-Hot Encoding).

Step 4. Final dataset

- Original dataset: **18 columns**
- After encoding: **26 columns** (because One-Hot added new columns for categories).
- Shape: **(10000, 26)** → 10,000 rows and 26 features ready for ML.

4) Data Transformation:

What is Data Transformation?

It means changing your **numeric features** into a suitable scale/format for machine learning models. Some algorithms (like Logistic Regression, KNN, Neural Nets) perform better when all numbers are on a **similar scale**.

Common Data Transformation techniques:

1. Standardization (Z-score scaling)

- Makes features have **mean = 0, std = 1**.
- Formula:

$$z = \frac{x - \mu}{\sigma}$$

2. Normalization (Min-Max scaling)

- Scales features to a **fixed range [0,1]**.
- Formula:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

3. Robust Scaling

 Robust scaling uses the **median** and **IQR (Interquartile Range: Q3–Q1)** instead of mean/std.

Formula:

$$X' = \frac{X - \text{Median}}{IQR}$$

5) Data Splitting:

✅ Data Splitting Completed!

Training set shape: (7000, 17)

Testing set shape: (3000, 17)

◆ Explanation in simple words:

- $X \rightarrow$ All features (Age, Income, LoanAmount, CreditScore, etc.).
 - $y \rightarrow$ Target variable (Default, i.e., whether the person defaulted or not).
 - train_test_split $\rightarrow$ Splits dataset:
 - **70% training data** $\rightarrow$ Used by the model to learn patterns.
 - **30% testing data** $\rightarrow$ Used to check how well the model performs on unseen cases.
 - stratify=y $\rightarrow$ Ensures the same proportion of default vs. non-default in both train and test sets.
-

6) Data Reduction:

📉 What happened in your case:

1. Original dataset:

- 100 samples with 123 features $\rightarrow$ high feature-to-sample ratio $\rightarrow$ prone to overfitting.

2. PCA transformation:

- Reduced the features from **123** $\rightarrow$ **93**.
- That's about a **24% reduction**, while still retaining **95.78% of the original information**.

3. How this helps:

- Your machine learning model now trains faster because it has fewer inputs.
- Irrelevant or redundant data has been removed without losing important patterns.
- It can help avoid overfitting by focusing on the major trends in the data.

📌 Important Notes:

- PCA does not always "improve accuracy" but it helps by:

- Reducing noise.
- Making the dataset smaller and easier to process.
- Often improving generalization to new data.
- In your case, even with only 100 samples, PCA reduced dimensions while keeping most of the variance, which shows that many features were either correlated or not contributing much independently.

✅ **Summary – For Your Dataset**

- **Why use PCA:** You had many features after encoding, and PCA helped reduce them while keeping the essential information.
- **What it did:** Reduced from 123 features to 93 → about 24% fewer features.
- **How much variance it kept:** 95.78% → almost all the important data patterns remain.
- **The benefit:** The dataset is now easier to process, less noisy, and faster to use for modeling without a significant loss of information.